ResQnet Map Engine — Complete Functionality & Role Logic

Technical and operational documentation of the integrated Map Engine: architecture, routing logic, geographic data flow, role-based behavior, navigation, hazards, closures, Safety Hubs, SOS, missions, GPS, route progress, rerouting, and UI interaction logic.

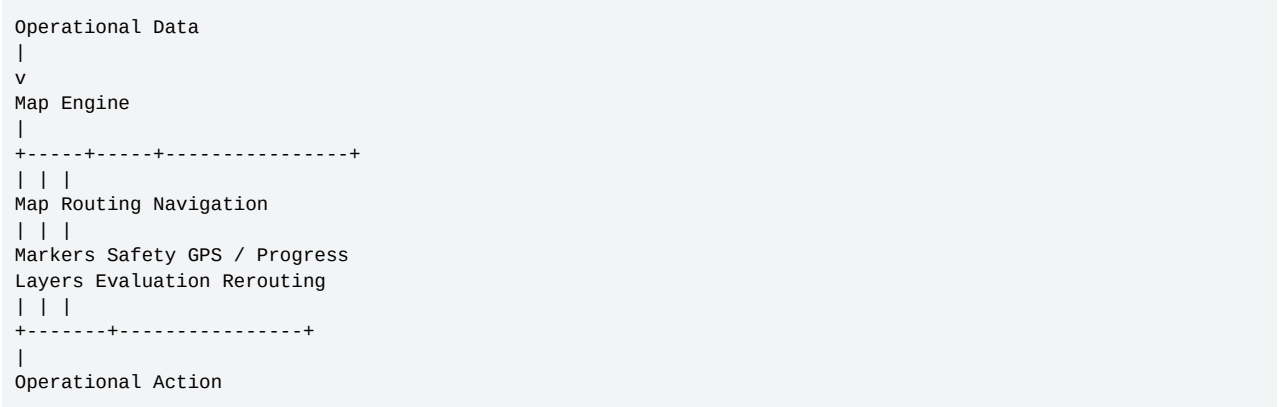
1. Document Scope

This document focuses **only on the ResQnet Map Engine** and the functionality/logic that the map contributes to the platform. It covers the existing routing-lab engine and its intended integration into ResQnet. General dashboard/business features are excluded unless they directly affect map behavior.

The Map Engine is located in the same project/repository level as ResQnet under **routing-lab/**. The integrated architecture uses the routing-lab implementation as the geographic/routing reference.

2. Purpose of the Map Engine

The Map Engine is the geographic and navigation foundation of ResQnet. Its purpose is to convert operational information—locations, hazards, road closures, incidents, responders, missions, and Safety Hubs—into geographic visualization, route decisions, and navigation behavior.



3. Core Architectural Responsibility

ResQnet owns: authentication, role permissions, persistent operational/business data, incidents, missions, responders, hazards, Safety Hubs, road business records, and operational workflows.

Map Engine owns: geographic rendering, road-following routing, route safety evaluation, detours, road-section geometry, GPS visualization, route progress, navigation arrows, off-route detection, rerouting, arrival behavior, and map interaction.

The target architecture has one authoritative geographic routing pipeline: ResQnet application state → Map Engine adapter → routing-lab services → OSRM → route safety/detour decision → GeoJSON → MapLibre.

4. Main Map Engine Components

Component / Service	Primary responsibility
Routing Service	Requests real road-following routes from OSRM and returns route candidates/geometry/metrics.
Route Safety Service	Evaluates route interaction with hazards and blocked-road geometry.
Routing Decision Service	Combines route candidates, safety evaluation, and detour logic into a route decision.
Detour Service	Generates and evaluates alternative routes around unsafe/blocked areas.
Road Section Service	Projects closure handles onto actual road geometry and slices a road LineString between gates.
Safety Hub Service	Handles Safety Hub eligibility, capacity/status logic, and entrance/location routing.
Route Progress Service	Projects GPS onto the route, calculates remaining distance/duration, and extracts remaining route geometry.
MapLibre Map Component	Renders base map, route, hazards, closures, arrows, markers, and interactive geographic layers.
GPS Pipeline	Uses browser geolocation watchPosition/clearWatch and maintains current location, accuracy, heading, speed, and status.

5. Geographic Coordinate Logic

The internal application representation is **{lat, lng}**. GeoJSON, MapLibre, and OSRM use **[lng, lat]**. Every conversion must preserve this convention.

```
Application:
{ lat, lng }

GeoJSON / MapLibre:
[ lng, lat ]

OSRM URL:
/route/v1/driving/{lng1},{lat1};{lng2},{lat2}
```

Coordinate reversal is a critical failure mode. A route, hazard, closure, hub, incident, responder, or GPS marker must not be sent to MapLibre in [lat, lng] order.

6. Map Engine Interaction Modes

Mode	Purpose
IDLE / EXPLORE	Inspect map, select/inspect geographic objects, pan and zoom without changing A/B.
PLACE_START	Explicitly place/update A.
PLACE_DESTINATION	Explicitly place/update B.
DRAG_START	Move A marker along the map/road context.
DRAG_DESTINATION	Move B marker.
SAFETY_HUB_AREA	Draw an operational Safety Hub area.
SAFETY_HUB_POINT	Mark a Safety Hub point.
FACILITY_AREA	Select/use a public facility area.
HAZARD_AREA	Draw a hazard polygon/area.
DANGER_AREA	Draw a danger/high-risk area.
BLOCK_ROAD	Select a full road for closure.
BLOCK_ROAD_SECTION	Select a road and place closure start/end gates.
POLYGON_EDIT	Edit an existing polygon.

In Explore mode, an empty click should inspect/select rather than change A/B. Pan, zoom, explicit Set Start/Set Destination, marker dragging, and explicit reset should remain distinct interactions.

7. A/B Routing Logic

```
A
|
+--> Start selection
|
+--> Current GPS (when explicitly activated)
|
+--> Draggable A marker
|
v
Route calculation
|
+--> OSRM candidates
|
+--> Safety evaluation
|
+--> Detour generation if required
|
v
Selected route
|
B
|
+--> Destination selection
```

```
+--> Safety Hub location/entrance
+--> Mission target
+--> Draggable B marker
```

A and B are separate from Closure Start/Closure End. Closure handles must never be reused as route endpoints.

8. Real Road-Following Routing

The Map Engine uses OSRM road-following routing rather than a straight line. The route request uses the driving profile and full route geometry; the integration preserves navigation steps and route metrics.

```
A + B
|
v
OSRM
|
+--> Route 1
+--> Route 2
+--> Alternatives (when available)
|
v
Route Safety Evaluation
|
v
Safe / Warning / Unsafe
|
v
Routing Decision
|
v
Final route
```

A route is not considered safe merely because OSRM can reach B. ResQnet evaluates the route against dynamic hazards and road closures before deciding which route to display/use.

9. Route Candidate and Decision Logic

The engine can work with multiple route candidates. The safety layer evaluates candidate interaction with hazard and closure geometry. When the primary route is unsafe or blocked, detour generation can construct alternative route candidates and evaluate them.

```
OSRM candidates
|
v
Hazard / closure evaluation
|
+----+------------------+
| |
SAFE UNSAFE/BLOCKED
| |
+--> selected Detour generation
|
v
new candidates
|
v
safety evaluation
|
v
valid detour
```

The final decision should prefer a safe/usable route according to the Map Engine's routing decision logic. A route must not be replaced by a synthetic straight line simply because a preferred route is unavailable.

10. Route Safety Logic

Hazards and closures are geographic constraints. Route safety uses spatial checks between route geometry and hazard/closure geometry. The audited engine uses bounding-box prefiltering, polygon/ray-casting or segment-intersection checks, and blocked-road proximity evaluation.

Conceptually, a route is evaluated as:

```
Route geometry
|
```

```
+--> intersects hazard?
|
+--> intersects blocked road?
|
+--> intersects blocked road section?
|
v
Safety classification
|
SAFE / WARNING / UNSAFE
```

11. Hazards

Supported hazard concepts include flood, waterlogging, high-risk zone, fire, debris, fallen tree, and structural damage. Hazard areas can be displayed on the map and incorporated into route evaluation.

Coordinator hazard authoring creates geographic geometry; the Map Engine then uses that geometry in route evaluation. Hazard creation is operational data, not decorative drawing.

12. Full Road Blocking

A full road closure means the route must not use the blocked road when the route actually intersects the blocked geometry. The engine evaluates the closure geographically rather than simply disabling arbitrary map pixels.

```
Coordinator
|
Block Road
|
Select road
|
Full road closure
|
Map Engine closure state
|
Route evaluation
|
Avoid blocked road
|
Alternative route
```

13. Road-Section Blocking

A road section is blocked by selecting the actual road geometry and defining two closure gates. The Road Section Service projects the handles onto the same road LineString and slices the geometry between the two positions.

```
Actual road LineString
------------------------------------------

Gate 1 Gate 2
| |
v v
-------------[■■■■■■■■■■■■]----------------
BLOCKED
ROAD SECTION

Result:
Only the geometry between Gate 1 and Gate 2 is blocked.
```

During gate dragging, the intended behavior is to preview geometry locally without repeatedly calling OSRM. The final confirmed section becomes part of route safety evaluation.

14. Safety Hubs

A Safety Hub is a geographic destination with operational eligibility. The Map Engine supports point, area, facility selection, optional entrance/access point, capacity, occupancy, verification, and active status.

The audited eligibility logic is based on active + verified + available capacity. Routing uses **entranceLocation when available, otherwise hub.location**.

```
Safety Hub
|
+--> active?
+--> verified?
```

```
+--> capacity > 0?
+--> occupancy < capacity?
|
v
Eligible destination
|
v
Route to entrance/location
```

15. Safety Hub Creation Workflow

```
+ Safety Hub
|
+--> Existing public facility
|
+--> Draw Safety Hub Area
|
+--> Mark Safety Hub Point
|
v
Optional entrance
|
v
Configuration
|
v
Validation
|
v
Confirm
|
v
Saved operational hub
```

Draft geometry must be separate from saved geometry. Creating or editing another hub must not accidentally move an already saved hub.

16. GPS / Live Location

The audited GPS pipeline uses browser geolocation watchPosition/clearWatch. GPS state includes status, latitude, longitude, accuracy, heading, speed, altitude, timestamp, and error information.

```
User enables Live Location
|
v
Permission request
|
+--> denied/unavailable/timeout -> ERROR
|
v
watchPosition
|
v
App live GPS state
|
+----+----+
| |
Map Navigation
marker state
```

GPS updates must not silently overwrite manual A/B routing unless the user explicitly uses current location for the relevant workflow. Responder mission navigation intentionally uses responder live GPS as A.

17. GPS Marker Logic

The Map Engine should update one persistent GPS marker rather than recreating markers for every fix. Geographic position uses MapLibre [lng, lat]. GPS accuracy can be displayed when useful.

The camera should not automatically recenter on every GPS update. Recenter/Center on Me should be explicit.

18. SOS Geographic Logic

```
Citizen presses SOS
|
```

```
v
Read current live GPS
|
+--> if unavailable, use approved fallback location
|
v
Create CRITICAL incident
|
v
Incident location fixed
|
v
Coordinator sees incident
|
v
Mission can be created
```

The SOS/incident marker and the responder's live GPS marker are separate geographic objects.

19. Mission Geographic Logic

The integrated design uses an explicit mission target coordinate for navigation. The mission may retain a road_id for legitimate ResQnet business relationships, but road_id is not the primary navigation destination.

```
Mission
|
+--> road_id (business context)
|
+--> target_lat
+--> target_lng
|
v
Responder navigation destination
```

The old method of deriving a mission target by averaging road/intersection endpoints must not be the authoritative navigation mechanism.

20. Responder Mission Navigation

```
Responder Live GPS
|
v
A
|
| OSRM + safety + closures
|
v
B = Mission Target
|
v
Road-following navigation
```

When the responder chooses Navigate to Mission, the Map Engine uses the responder's current GPS as A and the mission target as B. The mission destination remains stable while the responder position changes.

21. Navigation State Machine

```
IDLE
|
v
NAVIGATING
|
+--> OFF_ROUTE
| |
| v
| RE_ROUTING
| |
| +----> NAVIGATING
|
+--> ARRIVED
```

The audited navigation state machine uses GPS fixes to update route progress, remaining distance/duration, arrows, and arrival. Arrival is detected when the responder reaches the destination within the configured threshold.

22. Route Progress

Route progress projects the current GPS position onto the route LineString. The Map Engine then derives the remaining route geometry from the projected position onward.

```
Full route:
A ------------------------------ B
^
|
GPS

Remaining:
---------------------- B

Next GPS:
^
|
GPS

Remaining:
------------- B
```

This remaining geometry drives both progress information and navigation arrow placement.

23. Direction Arrow Logic

Direction arrows are generated from the remaining route geometry. The audited implementation generates arrows at approximately 60–100 m spacing (with integration-specific generation parameters) and calculates bearing from route segments.

```
Remaining LineString
|
v
Sample points
|
v
Segment bearing
|
v
Arrow features
|
v
MapLibre arrow layer
```

Arrows must use geographic coordinates and correct bearing. They should decrease as route progress increases, regenerate after rerouting, and clear on arrival.

24. Off-Route Detection

Off-route detection compares GPS fixes against the active route. The audited engine uses a route-distance threshold and requires repeated out-of-route fixes before declaring the responder off route.

```
GPS fix
|
v
Distance from route
|
+--> within threshold -> continue
|
+--> outside threshold
|
repeated fixes
|
v
OFF_ROUTE
|
v
Recalculate
```

The reroute keeps B as the mission destination while replacing A with the responder's current location.

25. Rerouting Logic

```
Current navigation
|
Road becomes blocked / hazard changes / responder off-route
|
```

```
v
Current GPS becomes new A
|
Mission destination remains B
|
v
OSRM candidates
|
v
Safety evaluation
|
v
New route decision
|
v
Remaining route + arrows updated
```

The engine should not repeatedly request OSRM for every tiny GPS movement. Rerouting is event/threshold driven.

26. Arrival Logic

Arrival is detected when the responder reaches the destination within the configured distance threshold. The navigation state changes to ARRIVED; navigation-only arrows are cleared and the mission can be presented as on-scene.

```
GPS
|
distance to B
|
< arrival threshold?
|
YES
|
ARRIVED
|
clear arrows
|
ON SCENE
```

27. Map Layers and Visual Priority

The integrated map uses deliberate layer ordering so route geometry remains visible while hazards, closures, and operational markers remain useful.

```
BASE MAP
|
HAZARD AREAS
|
ROAD SELECTION / EDITING
|
BLOCKED ROADS
|
CANDIDATE ROUTES (developer/diagnostic when enabled)
|
ROUTE CASING
|
ACTIVE ROUTE
|
DIRECTION ARROWS
|
OPERATIONAL MARKERS
|
GPS
|
A/B MARKERS
```

The exact style/layer ordering may follow the active MapTiler style, but route and navigation geometry must not be hidden behind hazard or UI layers.

28. Map UI Logic — No Overlap

Map functionality requires overlays, but the map remains the dominant geographic surface. UI elements must occupy defined regions instead of competing for the same pixels.

```
TOP
Header / Search / Status
```

```
SECONDARY
Operational status chips

LEFT
Navigation / mission card

RIGHT
Map controls

BOTTOM
Legend / contextual sheet

BOTTOM EDGE
Role navigation

MAP
Route + hazards + closures + markers
```

Do not solve overlap by randomly increasing z-index. Use layout regions, collapsible panels, responsive containers, and MapLibre camera padding.

29. Camera / Viewport Logic

Camera movement must be intentional. The Map Engine should not automatically recenter every time a user pans, zooms, activates a marking tool, or receives a GPS fix.

Explicit camera actions include Recenter, Center on Me, Focus Hub, and Fit Route. Route fitting should account for the UI regions so the geographic route remains visible without CSS-shifting the route itself.

30. Role-Based Map Functionality — Citizen

Function	Map behavior / logic
View map	Explore real geographic map with relevant hazards, closures, Safety Hubs, and route context.
GPS	Explicitly enable current location; show GPS marker and accuracy/context.
Set Start	Explicit tool sets A; return to Explore after placement.
Set Destination	Explicit tool sets B; return to Explore after placement.
Safe route	Run road-following routing and evaluate hazards/closures.
Safety Hub	View eligible hubs; route to entrance/location.
Evacuation	Provide a simple safe-evacuation action toward an eligible hub.
SOS	Capture current geographic location and create an emergency incident.
Navigation	Show route/progress/arrows where citizen navigation is enabled.
Restrictions	No coordinator dispatch, hazard authoring, or road-block authoring controls.

31. Role-Based Map Functionality — Coordinator

Function	Map behavior / logic
Operational map	See incidents, SOS, responders, missions, hazards, closures, and Safety Hubs.
Incident triage	Select an incident/SOS and inspect its geographic location.
Responder visibility	See responder locations and availability context where authorized.
Dispatch	Assign responder/mission and provide an explicit mission target.
Hazard authoring	Draw/create hazard areas; edit/delete where permitted.
Block road	Select and confirm an entire road closure.
Block road section	Select road and place Closure Start/End handles on actual geometry.
Safety Hub authoring	Create facility, point, or area hub with optional entrance and configuration.

Function	Map behavior / logic
Mission target	Select exact target location through the Map Engine; persist target coordinates.
Map tools	Use contextual authoring tools only when corresponding workflow is active.

32. Role-Based Map Functionality — Responder

Function	Map behavior / logic
Live location	Explicitly enable GPS; map current responder location.
Active mission	Display assigned mission target and operational context.
Navigate to Mission	A = responder GPS, B = explicit mission target.
Safe route	Route through current road/hazard/closure state.
Route progress	Project GPS onto route and display remaining distance/duration.
Direction arrows	Show arrows only along remaining route.
Off-route	Detect sustained deviation and trigger rerouting.
Rerouting	Current GPS becomes new A; mission target remains B.
Arrival	Detect destination arrival and clear navigation-only arrows.
Restrictions	No coordinator authoring tools unless separately authorized.

33. Cross-Role Operational Chain

```
CITIZEN
|
| SOS + location
v
INCIDENT
|
v
COORDINATOR
|
| triage + responder assignment
v
MISSION
|
| explicit target coordinates
v
RESPONDER
|
| live GPS
v
MAP ENGINE
|
+--> OSRM
+--> Hazards
+--> Road closures
+--> Road sections
+--> Route safety
+--> Detours
|
v
SAFE / USABLE ROUTE
|
+--> Progress
+--> Arrows
+--> Off-route
+--> Rerouting
+--> Arrival
|
v
ON SCENE / RESCUE
```

34. Dynamic Disaster Logic

The key value of the Map Engine is that route behavior can change when the operational geographic state changes.

```
Initial state
Responder --> Route A --> Incident

|
| Coordinator blocks road
v

Map Engine receives closure
|
v
Route A becomes unsafe/unusable
|
v
New route evaluated
|
v
Responder --> Detour --> Incident
```

The same principle applies when a hazard area is created/updated or when the responder goes off-route.

35. Data Ownership and Source of Truth

Persistent business data remains in ResQnet. The Map Engine consumes geographic state and computes geographic behavior.

Data / capability	Authoritative owner
Incidents / SOS	ResQnet operational data
Missions	ResQnet operational data; explicit target coordinates for navigation
Responders	ResQnet operational data + live GPS state
Hazards	ResQnet operational data / Map Engine geographic representation
Safety Hubs	ResQnet operational data + Map Engine geographic behavior
Road business records	ResQnet; not a route generator
Routing	routing-lab Map Engine + OSRM
Route safety	routing-lab route safety logic
Detours	routing-lab detour logic
Route progress	routing-lab progress logic
Navigation arrows	routing-lab navigation/arrow logic
Map rendering	Integrated MapLibre Map Engine

36. Legacy Map Replacement Logic

The new architecture removes the old competing map/routing behavior while preserving legitimate ResQnet business data.

```
OLD
ResQnet
|
Dijkstra
|
Synthetic / virtual road graph
|
Old map

NEW
ResQnet business state
|
Map Engine Adapter
|
routing-lab
|
OSRM + route safety + detours
|
MapLibre
|
Real geographic map
```

Roads and intersections may remain when required for legitimate business relationships, field surveys, or operational records. They must not become a second production route-generation engine.

37. UI Generation Rule for Missing Map Functions

If a required map function does not have an appropriate ResQnet UI, the function must receive a purpose-built interface rather than being omitted or exposed only through developer tools.

Examples include Coordinator Block Road Section, Responder Navigate to Mission, and Citizen Safe Evacuation to an eligible hub. New UI must match ResQnet's existing visual language and must not overlap other controls.

38. No-Overlap Map UI Rules

• Every overlay belongs to a defined region.

• No two independent components should occupy the same pixel space.

• Active navigation has priority over promotional/secondary content.

• Legends are collapsible and role-aware.

• Map controls remain accessible and do not cover route-critical information.

• Bottom navigation has a reserved region.

• Responsive layouts must work on desktop, tablet, and mobile.

• MapLibre camera padding is used to keep route geometry visible around UI; CSS must not move geographic objects.

39. End-to-End Functionality by Role

```
CITIZEN
Explore
-> GPS
-> Safe Route
-> Safety Hub
-> Evacuation
-> SOS

COORDINATOR
Incident
-> Inspect location
-> View responders
-> Dispatch
-> Create hazard
-> Block road/section
-> Manage Safety Hub

RESPONDER
Live GPS
-> Receive mission
-> Navigate
-> Follow route
-> Progress/arrows
-> Off-route detection
-> Reroute
-> Arrive
```

40. Failure and Edge-Case Logic

Situation	Expected Map Engine behavior
GPS permission denied	Show explicit GPS error state; do not pretend GPS is active.
GPS unavailable	Keep manual A/B workflow available where permitted.
Invalid route geometry	Do not render fake route; surface routing error.
No safe route	Use the engine's valid fallback/decision behavior; never silently draw a straight line.
Road becomes blocked	Re-evaluate active route and detour when required.
Responder off route	After configured threshold/confirmation, reroute from current GPS.

Situation	Expected Map Engine behavior
Safety Hub full/closed/unverified	Do not treat it as an eligible evacuation destination.
Map pan during marking	Preserve viewport; do not auto-jump camera.
Saved hub while drawing another	Keep saved geometry immutable; edit draft separately.
Route arrival	Transition to ARRIVED and clear navigation-only arrows.

41. Functional Acceptance Checklist

1. ■ Real road-following A/B route is visible.
2. ■ OSRM route geometry is valid LineString with [lng, lat] coordinates.
3. ■ Route steps are preserved and available to navigation UI.
4. ■ Direction arrows render on remaining route.
5. ■ Arrows decrease with progress and regenerate after rerouting.
6. ■ Hazards affect route safety.
7. ■ Full road closures affect route safety.
8. ■ Road-section closures affect route safety.
9. ■ Safety Hub eligibility and entrance/location routing work.
10. ■ Citizen GPS and SOS workflow work.
11. ■ Coordinator incident/dispatch and map authoring work.
12. ■ Responder live GPS and mission navigation work.
13. ■ Off-route detection and rerouting work.
14. ■ Arrival detection clears navigation-only arrows.
15. ■ Map markers remain geographic and correctly anchored.
16. ■ Map UI does not overlap.
17. ■ Role-specific controls are enforced.
18. ■ Missing required functions receive appropriate UI.
19. ■ One authoritative routing engine is used.
20. ■ Old competing map/routing system is removed after parity.
21. ■ Business tables such as roads are not incorrectly deleted.

42. Core Takeaway

The ResQnet Map Engine is not a decorative map. It is the geographic execution layer connecting disaster information to movement.

For the Citizen, it answers where they are, what routes are usable, and where an eligible Safety Hub can be reached. For the Coordinator, it provides the geographic control surface for incidents, responders, hazards, closures, road sections, Safety Hubs, and mission targets. For the Responder, it converts a mission into live GPS navigation that reacts to hazards, closures, route progress, off-route movement, and arrival.

```
INFORMATION
|
v
GEOGRAPHIC STATE
|
v
MAP ENGINE
|
+----+-----------------------------+
| |
ROUTE DECISION NAVIGATION
| |
```

```
Safe / usable route GPS / progress
| arrows / reroute
+------------------+---------------+
|
v
FIELD ACTION
```

43. Reference Implementation

The Map Engine reference implementation is the existing **routing-lab/** functionality integrated into ResQnet through an application adapter. The integration should preserve the proven routing, safety, detour, road-section, Safety Hub, progress, GPS, and navigation logic rather than creating parallel implementations.

This document intentionally focuses on Map Engine behavior and role-based geographic functionality rather than general ResQnet dashboard/business features.